

MATERIAL DE APOIO

ELEMENTOS DE BLOCO

Os **elementos de bloco** são todos aqueles que ocupam 100% do espaço horizontal disponível, independentemente da largura do seu conteúdo. Basicamente cada elemento de bloco irá ocupar **uma linha** dentro do documento. Logo, o próximo elemento será renderizado abaixo e não ao lado.

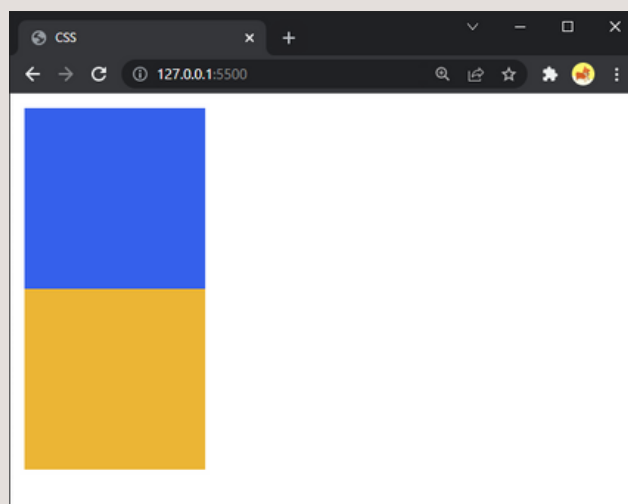
São exemplos de elementos de bloco nativos: `<div>`, `<section>`, `<p>`, elementos de título (`<h1>` até `<h6>`), entre outros. Nesse tipo de elemento, podemos setar tamanhos personalizados para o bloco, ou seja, é possível definir **height** e **width** ainda que o elemento ocupe a linha inteira.

HTML

```
<div id="square1"></div>
<section id="square2"></section>
```

CSS

```
#square1 {
  background-color: rgb(53, 96, 235);
  height: 100px;
  width: 100px;
}
#square2 {
  background-color: rgb(235, 181, 53);
  height: 100px;
  width: 100px;
}
```





ELEMENTOS INLINE

Já os **elementos de linha (inline)** ocupam apenas o **espaço necessário** para que o seu conteúdo seja exibido. Logo, não existe nesses elementos o conceito de uma linha para cada elemento, de modo que vários elementos inline junto serão renderizados lado a lado na página.

São exemplos de elementos inline: `<span>`, `<a>`, `<em>`, `<strong>`, `<img>`, entre outros. Uma grande diferença deste tipo de elemento para um elemento de bloco é a **incapacidade** de definirmos **a altura e a largura** do elemento. Ou seja, o tamanho será definitivamente apenas o necessário para o conteúdo, não sendo possível usar as propriedades `height` e `width`.

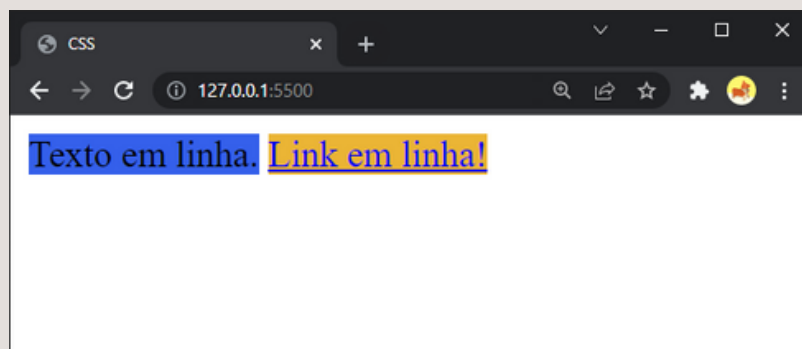
HTML

```
<span>Texto em linha.</span>
<a href="#" id="link">Link em linha!</a>
```

CSS

```
span {
  background-color: rgb(53, 96, 235);
  height: 100px;
  width: 100px;
}

#link {
  background-color: rgb(235, 181, 53);
  height: 100px;
  width: 100px;
}
```





DISPLAY

A propriedade **display** permite modificar o **modo de exibição** dos elementos na página. Ou seja, é possível modificar um elemento de bloco para se comportar e ser renderizado em linha, assim como fazer um elemento de linha se tornar um bloco.

Os valores de display iniciais dos elementos seguem seus padrões, sendo que a tag <div> recebe o valor **block** e a tag recebe o valor **inline**, por exemplo. Mas também existem outras variações além de block e inline: **inline-block**, **flex** e **grid**.

```
#square1 {  
  display: block;  
  display: inline;  
  display: inline-block;  
  display: flex;  
  display: grid;  
}
```

BLOCK

O display **block** faz com que o elemento se torne de bloco. Isto é, independentemente de qual tag é o componente, ele ocupará uma **linha** inteira para ser renderizado – 100% da largura disponível.





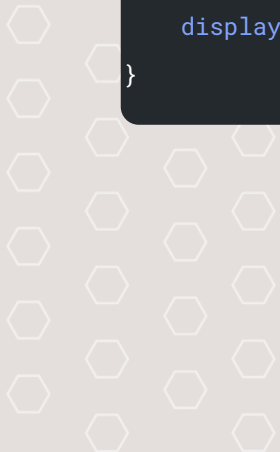
INLINE

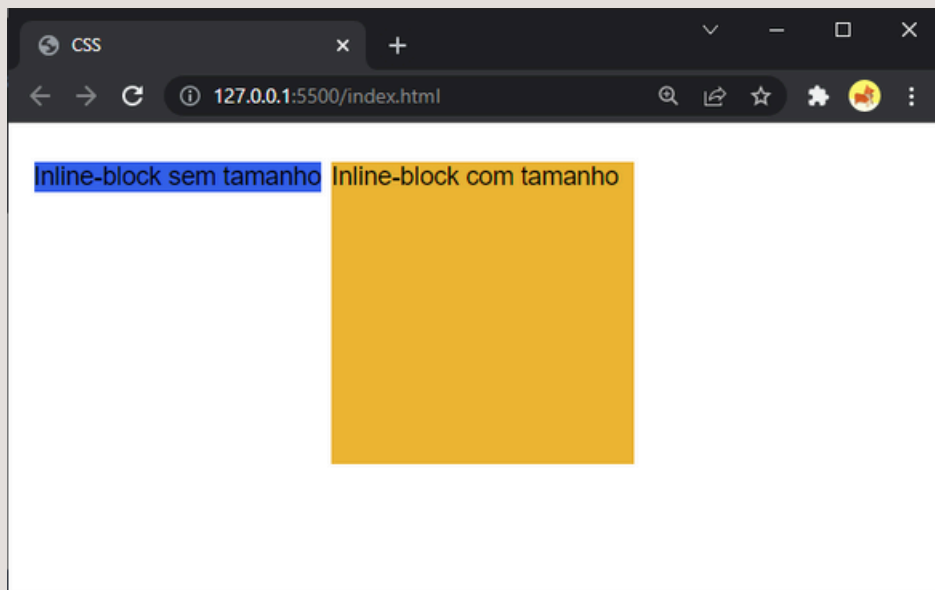
Já o display **inline** torna o elemento um componente de linha, de modo que a renderização levará em conta apenas o tamanho necessário do seu conteúdo. Neste caso, é importante lembrar que o elemento perde a capacidade de setar width e height pois estas propriedades são ignoradas pela renderização.

INLINE-BLOCK

O display inline é uma alternativa para posicionar elementos de bloco lado a lado dentro da página. Porém, a limitação de tamanhos personalizados é um problema. Por isso, o display **inline-block** é uma alternativa interessante pois torna **inline** o elemento mas mantém a **capacidade de se definir tamanhos** personalizados.

```
#square2 {  
  background-color: rgb(235, 181, 53);  
  height: 120px;  
  width: 120px;  
}  
  
#square1,  
#square2 {  
  display: inline-block;  
}
```





FLEX

O display **flex** é uma propriedade relativamente recente no CSS e foi criado devido a grande dificuldade de criação de **layouts responsivos** – que se adaptam a diferentes tamanhos de tela. Atualmente o display flex é uma das propriedades mais usadas para a **disposição de elementos nas páginas**.

Com o flex, os elementos passam a estar no modelo **flexbox**, em que um componente **container** agrupa outros componentes filhos (**flex-itens**). A organização dos itens pode ser direcionada em linha ou em colunas. Também há outras propriedades dentro do contexto flex que auxiliam na disposição dos elementos de uma forma mais dinâmica.

GRID

O display **grid** (também chamado de **Grid Layout**) também é um recurso recente e criado principalmente para a construção de layouts de páginas responsivas. Assim como no flexbox, no grid também existe o conceito de **container** que possui diversos **itens**.



A diferença para o flexbox está no fato de que o grid é **multidimensional**. No flex, o container trabalha com a direção de linha ou coluna. Já no grid, **linhas e colunas** podem ser combinadas no mesmo container, formando um layout mais complexo. Por isso, sua utilização está mais para a **construção de layouts**, enquanto o flex está para a disposição de elementos. Ainda assim, é possível usar ambos no mesmo projeto.

REFERÊNCIAS

1. [display | CSS-Tricks](#)
2. [Elementos block-level - HTML: Linguagem de Marcação de Hipertexto | MDN](#)
3. [Elementos Bloco e Em Linha - HTML e CSS na Prática - Guilherme Müller - Web Developer](#)
4. [CSS Grid e Flexbox - Quando utilizar? | Felipe Fialho - Front-end Developer](#)



BONS ESTUDOS!

